Problem # 01

The government has launched an investigation into historical records. Unfortunately, the dates in its database are so mixed up that even events from the past appear to have happened in the future! The officials need your help to put everything in chronological order.

Develop a C++ program that takes the day, month, and year as input for at least 5 dates and sorts them in ascending chronological order using Quick Sort.

Requirements:

- Take at least 5 dates as input from the user (day, month & year).
- Sort the dates by comparing the year first, followed by the month and then the day.
- Display the original and sorted dates in the format DD/MM/YYYY.

Problem # 02

Solve problem # 01 using merge sort.

Problem # 03

Solve problem # 01 using radix sort.

Problem # 04

The company has a serious problem: the CEO is sitting next to the office printer, the CTO is sharing a corner with the photocopier, and an EMP has somehow claimed a seat with a perfect view of the CEO's office! The management has decided that seating arrangements must reflect the corporate hierarchy. After all, apparently even office chairs have a chain of command! The facilities management team has hired you to restore corporate order. Develop a C++ program that sorts the employees' desk arrangements using the Quick Sort algorithm, based on their designations and the following priority order:

1. **CEO** (Chief Executive Officer) — Highest Priority
2. **CTO** (Chief Technology Officer)
3. **CFO** (Chief Financial Officer)
4. **VP** (Vice President)
5. **MGR** (Manager)
6. **EMP** (Employee) — Lowest Priority

The higher the designation, the closer the employee should be seated to the corner office.

Initial arrangement of desks from left to right:

EMP, CFO, MGR, EMP, VP, CTO, MGR, CEO

Requirements:

- Implement the Quick Sort algorithm to arrange the employees in descending order of designation priority, from highest to lowest.
- Display the initial and final seating arrangements.
- Use the designation priority given above to determine the correct sorting order.

Expected Output:

CEO, CTO, CFO, VP, MGR, MGR, EMP, EMP

Problem # 05

Solve problem # 04 using merge sort.

Problem # 06

Solve problem # 04 using radix sort.

Problem # 07

A gaming café has just finished hosting a tournament, but its staff has managed to record the players' scores in completely random order. The player with 150 points is sitting below someone with 40 points on the leaderboard, and everyone is accusing the organizers of rigging the tournament! The café owner wants you to sort out this mess before the losing players start demanding a rematch.

Develop a C++ program that stores the scores and names of at least 8 players in a singly linked list and sorts them in descending order using the Quick Sort algorithm.

Requirements:

- Create a singly linked list containing the names and scores of the players entered by the user.
- Display the original linked list before sorting.
- Implement Quick Sort to sort the linked list in descending order.
- Display the sorted linked list after sorting.
- Perform the sorting directly on the linked list. Copying the values into an array for sorting is not allowed.

Sample Input:

```
Ali(120) -> Ahmed(50) -> Sara(80) -> Hamza(150) -> Ayesha(40) -> Usman
                (100) -> Zoya (60) -> Bilal (90) -> NULL
```

Sample Output (Sorted):

```
Ayesha(40) -> Ahmed(50) -> Zoya(60) -> Sara(80) -> Bilal(90) -> Usman (100)
                    -> Ali (120) -> Hamza (150) -> NULL
```

Problem # 08

Solve problem # 07 using merge sort.

Problem # 09

Solve problem # 07 using radix sort.